

操作系统，2026

第7讲 处理机调度 & 死锁问题

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

处理机调度

02

Linux/Windows的处理机调度

02

死锁问题

处理机调度

- 处理机是计算机系统中最重要的资源
- 在多道程序设计系统中，通常会有多个进程（或线程）竞争处理机资源，因此操作系统必须选择要运行哪一个进程（或线程），为其分配处理机
- 完成选择工作的操作系统代码称为调度程序(scheduler)

处理机调度的时机

1. 在创建一个新进程后
 2. 当一个进程运行完毕，或由于某种错误而终止运行时
 3. 当一个进程转入等待状态
 4. I/O中断发生时
 5. 时钟中断发生时
- 2、3两种情况下必须执行处理机调度程序。根据其他情况下是否执行调度程序，可以把调度的方式分为两类：
- 非抢先式调度
 - 抢先式调度

非抢先式(non_preemptive)调度

- 调度程序一旦把处理机分配给某进程后便让它一直运行下去，直到进程完成或进程因等待某事件而阻塞时，才把处理机分配给另一个进程
- 实现简单，系统开销小，主要适用于批处理系统，不适用于分时系统和实时系统

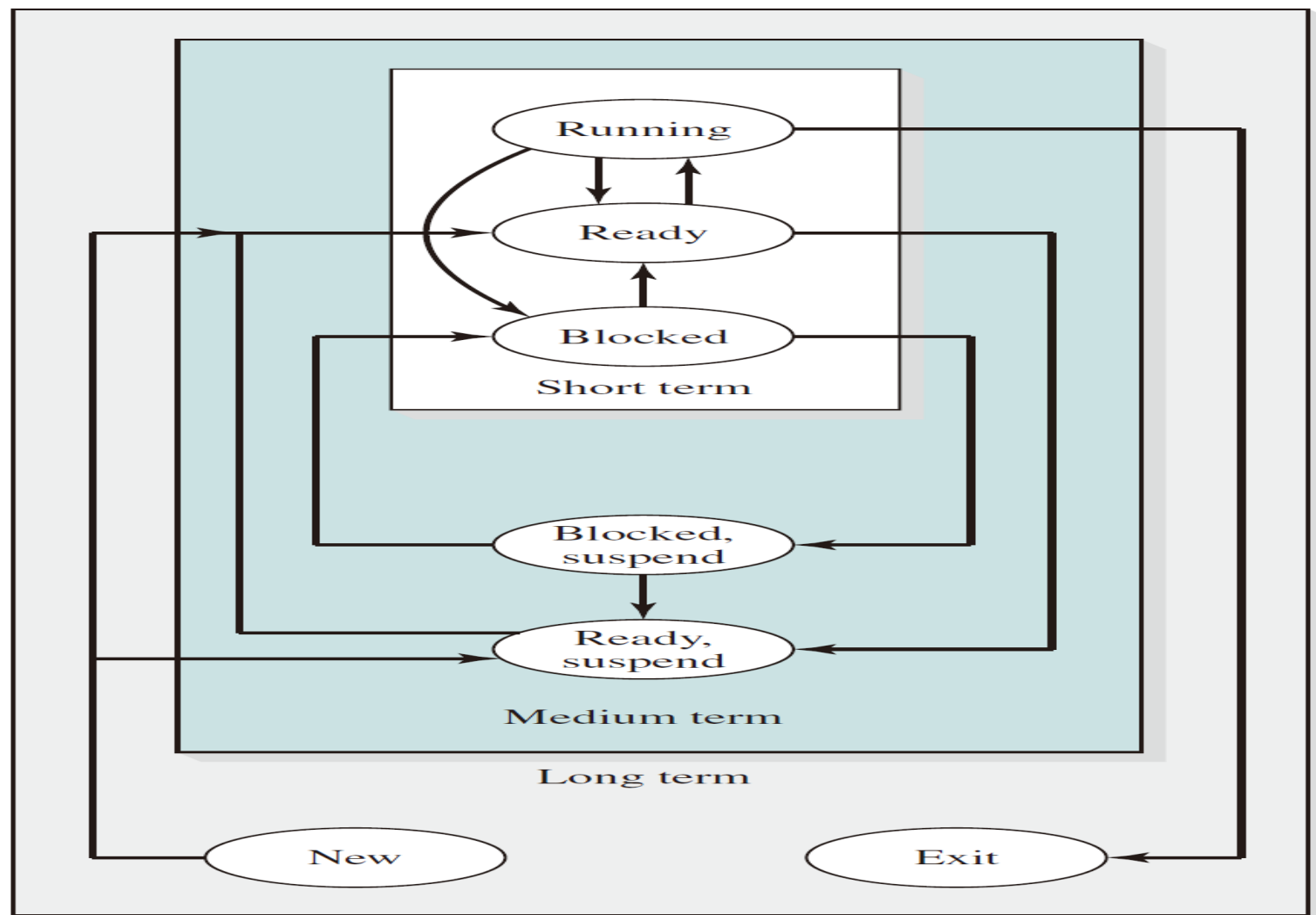
抢先式(preemptive)调度

- 当进程正在处理机上运行时，调度程序可以根据规定的原则抢先分配给此进程的处理机，并将其移入就绪列队，选择其他进程运行
- 主要适用于分时系统和实时系统
- 抢先原则有两种：
 - 优先级原则：高优先级进程可抢先低优先级进程
 - 时间片原则：当运行进程的时间片用完后被抢先

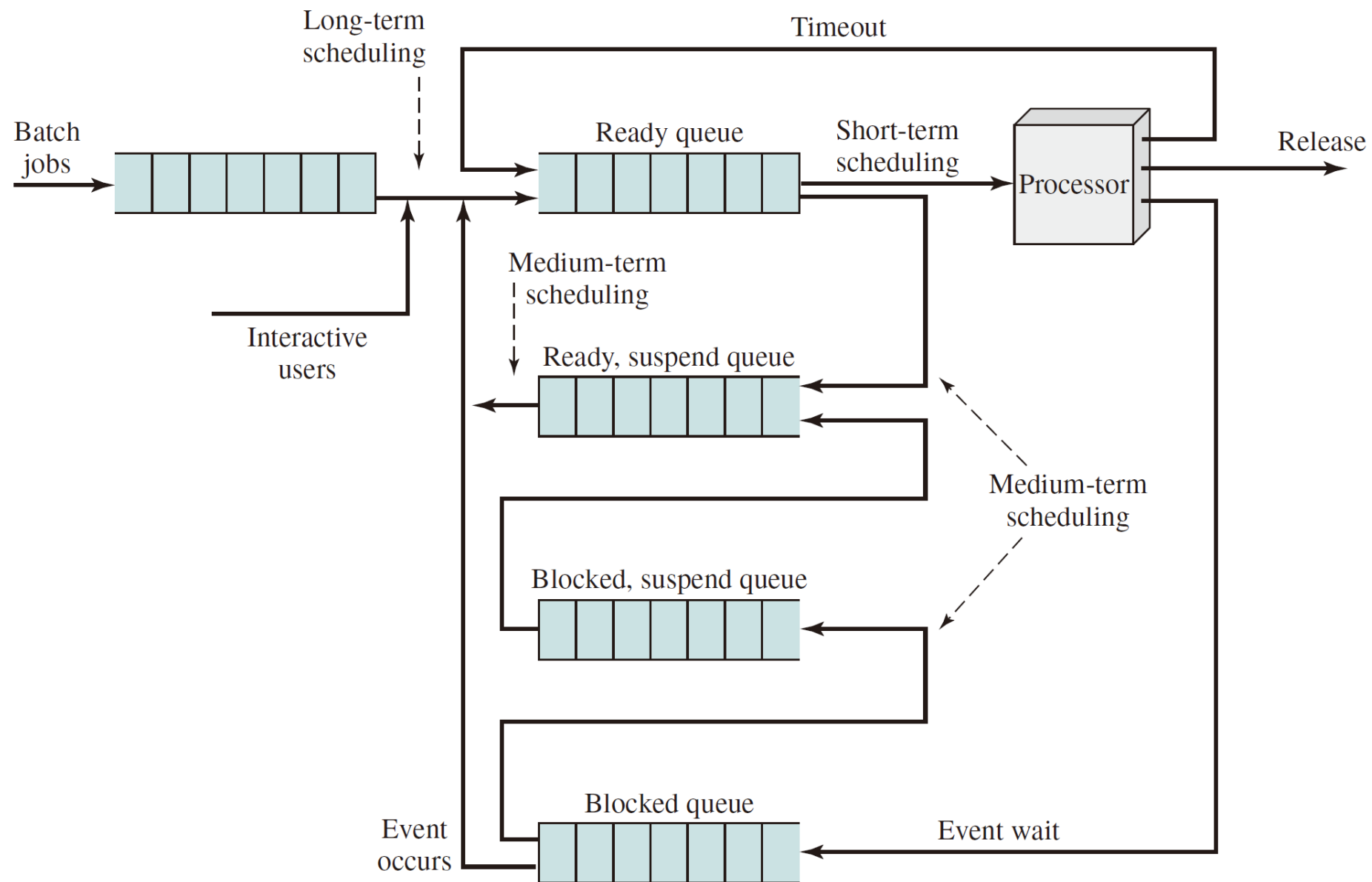
调度的层次

- **长程调度**：即**作业调度**，也称为宏观调度、高级调度。从用户工作流程的角度，一次提交的若干个流程，其中每个程序按照流程进行调度执行。时间尺度通常是分钟、小时
- **中程调度**：即**内外存交换**，也称为中级调度。从存储器资源的角度看，将进程的部分或全部换出到外存上，将当前所需部分换入到内存
- **短程调度**：即**进程调度(线程调度)**，也称为微观调度、低级调度。从CPU资源分配的角度看，CPU要经常选择就绪进程或线程进入运行状态。时间尺度通常是毫秒级。因为执行频繁，要求在实现时达到高效率

调度的层次



调度的层次

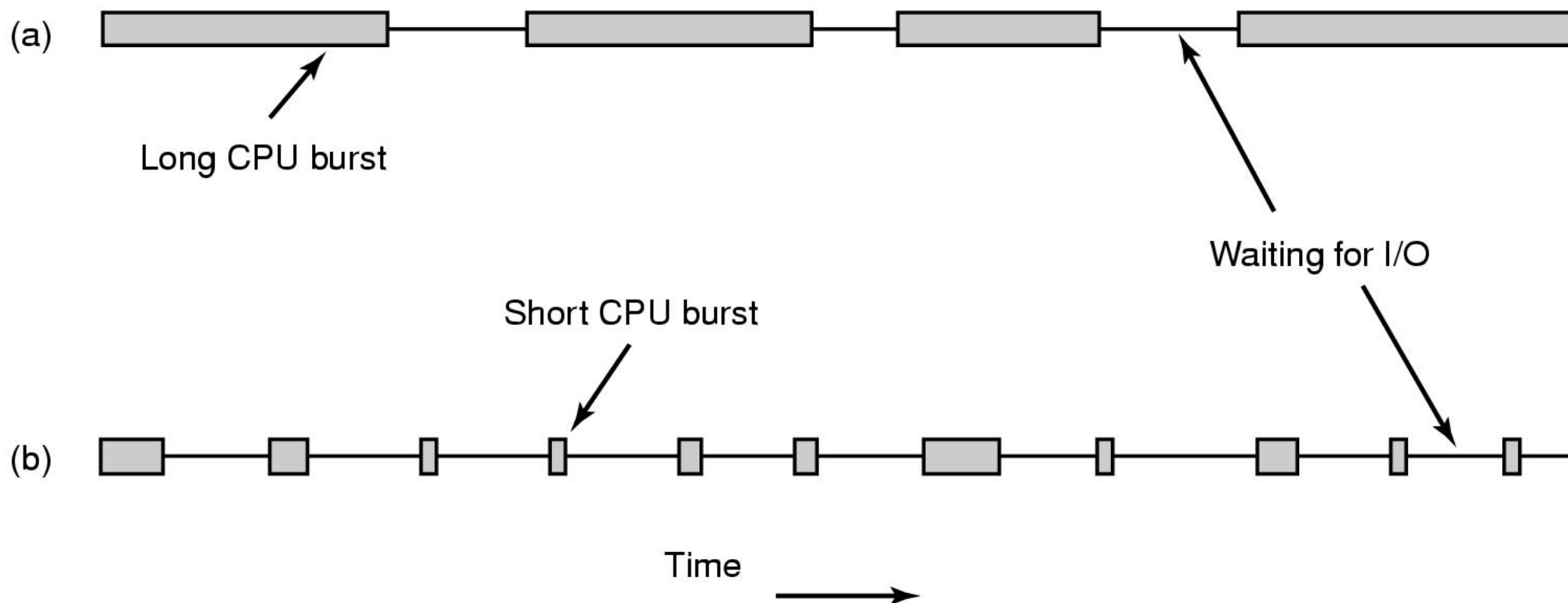


调度算法分类

- 不同的环境需要不同的调度算法，因为不同的应用领域有不同的目标
 - 批处理系统
 - 交互式系统
 - 实时系统

进程的行为

■ 计算密集型 vs I/O密集型



调度算法的目标

- 处理机调度算法的优劣，可以从多个方面判断。有些目标取决于环境，有些适用于所有情形
- 所有系统
 - 公平——给每个进程公平的CPU份额
 - 效率——保持系统的所有部分都忙碌

调度算法的目标

■ 批处理系统

- 吞吐量——每小时最大作业数
- 周转时间——从提交到终止间的最小时间
- CPU利用率——保持CPU忙碌

■ 交互式系统

- 响应时间——发出命令到得到响应之间的时间。快速响应请求

■ 实时系统

- 保证截止时间——避免丢失数据、发生灾难

批处理系统中的调度

- 批处理系统主要关注高吞吐量、低平均周转时间和高CPU利用率，目标是全局优化而不是提高单个任务的性能
- 批处理调度算法以非抢先为主，以避免上下文切换开销
- 典型的批处理调度算法
 - 先来先服务
 - 最短作业优先
 - 最高响应比优先
 - 最短剩余时间优先

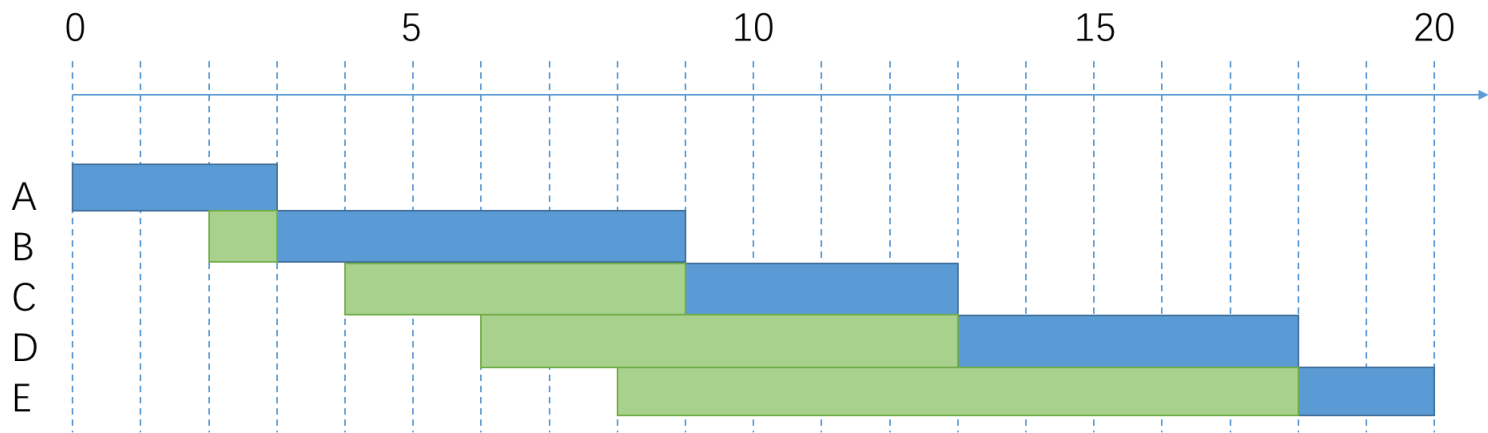
先来先服务

- 先来先服务(**FCFS**, First Come First Served)
- 按进程的提交或变为就绪的先后顺序进行调度
- 特点:
 - 实现简单、非抢先式
 - 有利于长作业而不利于短作业
 - 有利于CPU密集的作业而不利于I/O密集的作业

先来先服务

■ 周转时间：A3、B7、C9、D12、E12，平均8.6

进程	到达时刻	所需运行时间
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



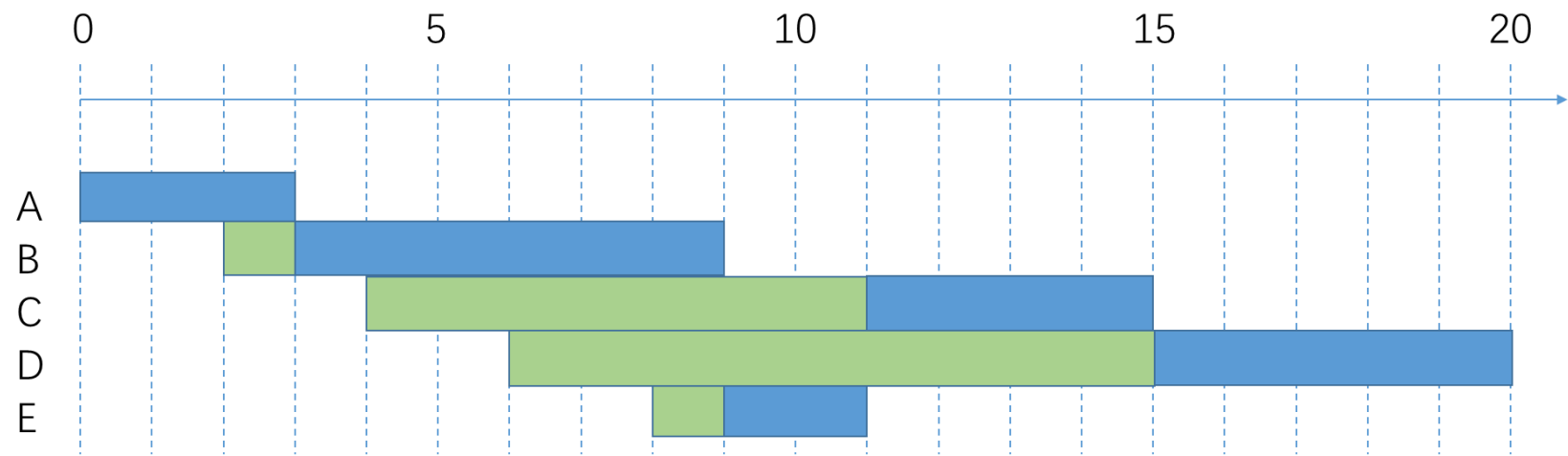
最短作业优先

- 最短作业优先(SJF, Shortest Job First)
- 预计执行时间短的作业(进程)优先分派处理机。它的设计目标是减少进程的平均周转时间
- 特点:
 - 与FCFS相比, 改善了平均周转时间
 - 对长作业不利
 - 难于预计作业的执行时间

最短作业优先

■ 周转时间：A3、B7、C11、D14、E3，平均7.6

进程	到达时刻	所需运行时间
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



最高响应比优先

- 最高响应比优先(HRRN, Highest Response Ratio Next)

- 响应比公式:

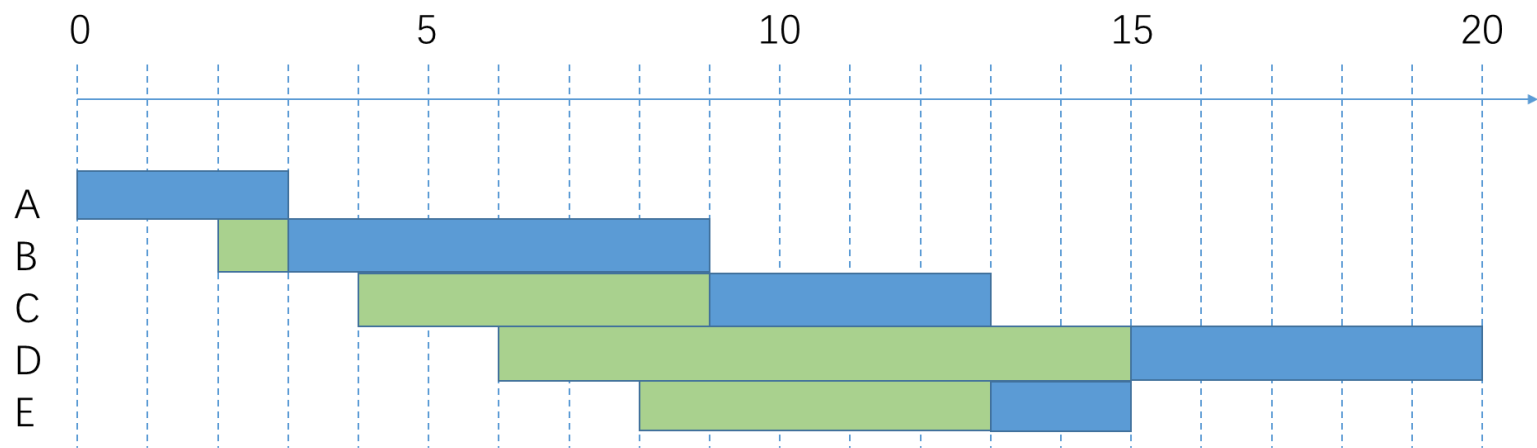
$$\text{响应比} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = 1 + \frac{\text{等待时间}}{\text{要求服务时间}}$$

- 介于FCFS与SJF之间的折中算法, 既考虑作业等待时间又考虑作业运行时间, 既照顾短作业又不使长作业等待时间过长

最高响应比优先

■ 周转时间：A3、B7、C9、D14、E7，平均8

进程	到达时刻	所需运行时间
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



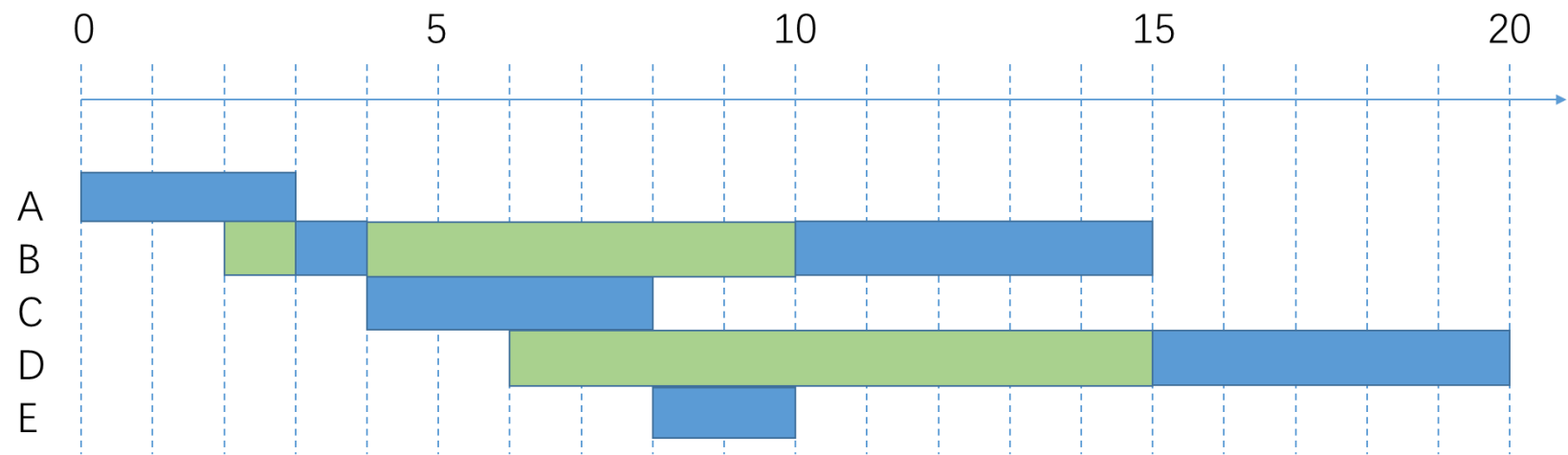
最短剩余时间优先

- 最短剩余时间(SRT, shortest remaining time)
- 最短作业优先算法的抢占式版本
- 调度程序总是选择剩余时间最短的进程运行。如果新的进程比当前运行的进程需要更少的时间，当前进程将被抢先而运行新的进程
- 特点：可使新的短作业获得良好的服务

最短剩余时间优先

■ 周转时间：A3、B13、C4、D14、E2，平均7.2

进程	到达时刻	所需运行时间
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



交互式系统中的调度

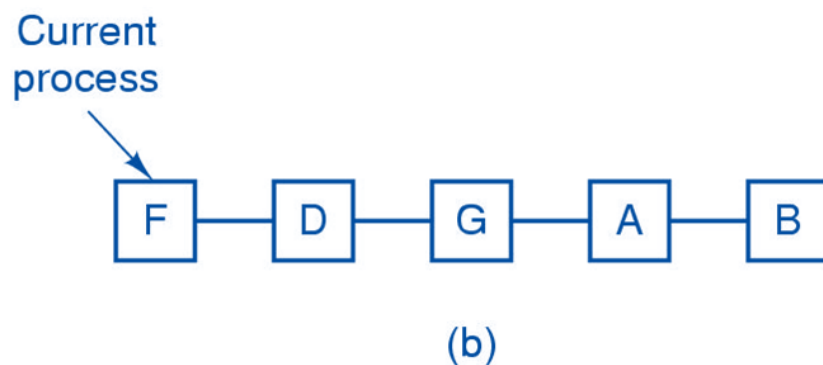
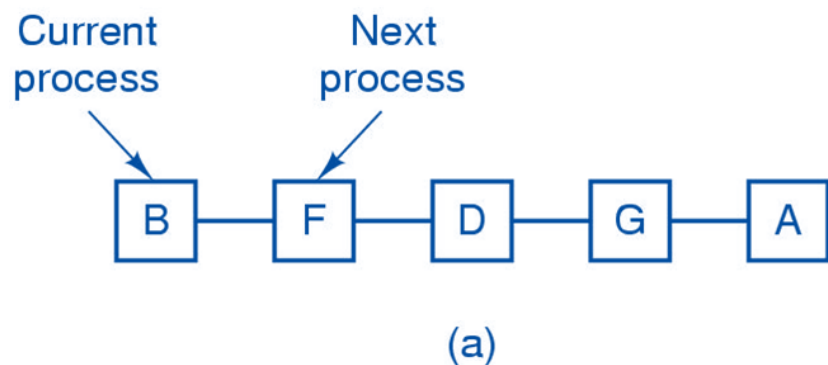
- 交互式系统中，调度的核心目标是保证较短的**响应时间**和进程/线程间的**公平性**，以优化用户体验
- 普遍采用抢先式调度算法，常见的调度算法包括：
 - 时间片轮转调度
 - 优先级调度
 - 多级反馈队列
 - 最短进程优先

时间片轮转调度

- 时间片轮转(RR, round robin)
- 每个进程被分配一个时间段, 称为时间片(quantum), 即允许该进程运行的时间
- 如果在时间片结束时该进程还在运行, 则将剥夺CPU并分配给另一个进程; 如果该进程在时间片结束前阻塞或结束, 则CPU立即进行切换

时间片轮转调度

■ 调度程序需要维护一张可运行进程列表



■ 特点：简单、公平；隐含所有进程同等重要的假设

时间片轮转调度

- 时间片长度对调度性能的影响
 - 时间片过短会导致过多的进程切换，降低CPU效率
 - 时间片过长会导致交互请求的响应时间变长
 - 将时间片长度设为20ms ~ 50ms通常是一个比较合理的折衷

优先级调度

- 优先级调度(Priority Scheduling)
- 每个进程被赋予一个优先级，允许优先级最高的可运行进程先运行
- 优先级的确定有静态和动态两种方式
 - 静态优先级：在进程创建时指定优先级，在进程运行时优先级不变
 - 动态优先级：在进程创建时设置一个优先级，但在其生命周期内优先级可以动态变化。如等待时间长优先级可改升高

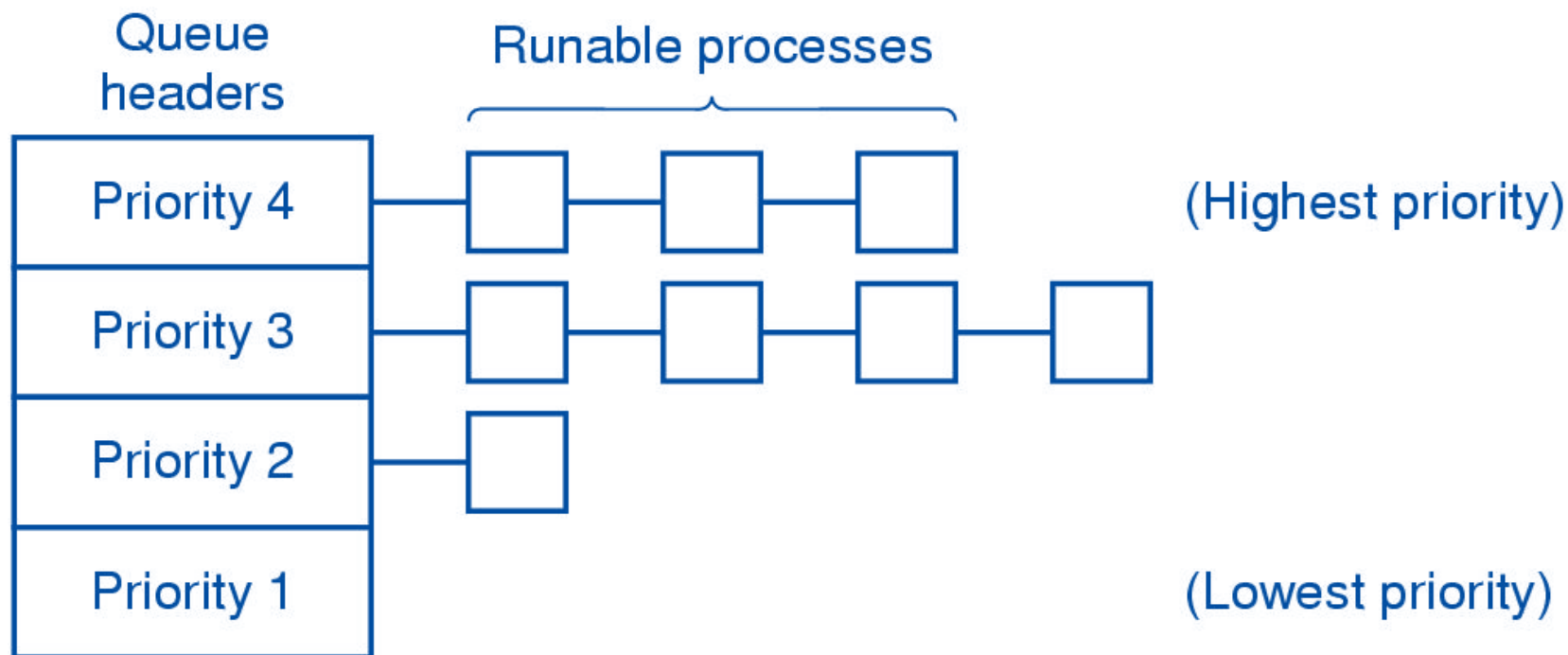
优先级调度

■ 动态优先级

- I/O密集型的进程需要CPU时，应立即分配给它CPU，以便启动下一个I/O请求，这样就可以在另一个进程计算的同时执行I/O操作
- 使I/O密集型进程获得较好服务的简单算法：将其优先级设为 $1/f$ ， f 为该进程在上一时间片中计算时间所占的比重

优先级调度

- 优先级调度解决先选哪个进程运行的问题，同一优先级内一般采用时间片轮转调度



多级反馈队列

- 多级反馈队列(Multilevel Feedback Queue, MLFQ)改进是优先级调度+时间片轮转的
- 为CPU密集型的进程设置较长的时间片比频繁地分给它们很短的时间片要高效，但是时间片过长又会影响到响应时间
- 解决方法：设立优先级类——高优先级的队列时间片短、低优先级的队列时间片长

多级反馈队列

■典型规则：

- 新进程 → 高优先级队列
- 时间片用完 → 降低优先级
- 经常阻塞 → 提高优先级

■MLFQ通过这一简单规则能够自动识别进程是交互式还是CPU密集型，为什么？

■多级反馈队列调度算法使CPU密集型的进程优先级低，时间片长，交互式进程优先级高，时间片短，以其达到快速响应和高吞吐量的目的

最短进程优先

- 最短进程优先(shortest process next)
- 最短作业优先的交互式版本
- 交互式进程的模式：等待命令、执行命令、等待命令、执行命令...
- 如果将每一条命令的执行看成是一个独立的“作业”，则可以通过首先运行最短的作业使响应时间最短

最短进程优先

- 对于每个可运行的进程，可以根据过去的行为对其执行时间进行预测，并执行估计运行时间最短的那个进程
- 假设某终端上的一条命令估计运行时间为 T_0 ，当前实际测量值为 T_1 ，则新的估计时间为 $aT_0 + (1-a)T_1$ ， $0 \leq a \leq 1$ 。一般取 $a=0.5$
- 这种通过当前测量值和先前估计值进行加权平均而得到下一个估计值的技术称为老化(aging)

实时系统中的调度

- 实时系统是时间起主导作用的系统。外设给计算机一个刺激，计算机必须在一个确定的时间范围内恰当地做出反应，即保证在**截止时间**之前响应
- 在实时系统中，迟到的响应即使正确，也和没有响应一样糟糕
- 实时系统可以分为硬实时和软实时
 - 硬实时(HRT, Hard Real Time)——必须满足绝对的截止时间
 - 软实时(SRT, Soft Real Time)——偶尔超过时间限制是可以容忍的

实时系统中的调度

- 实时系统中的事件可以按响应方式分为周期性事件(以规则的时间间隔发生)和非周期性事件(发生时间不可预知)
- 一个系统可能要响应多个周期性事件流和非周期性事件流
- 根据每个事件需要处理时间的长短，系统有可能无法处理完所有事件

实时系统中的调度

- 如果有m个周期性事件，事件i以周期 P_i 发生，并且需要 C_i 秒CPU时间处理，那么在单处理机系统中可以处理负载的条件为

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- 满足该条件的实时系统称为是可调度的

实时系统中的调度

- 基于优先级的立即抢先(immediate preemption)调度策略是实时系统的主要调度策略
- 常用调度算法
 - 速率单调调度
 - 最早截止时间优先
 - 最小裕度优先

速率单调调度

- 速率单调调度(RMS, Rate Monotonic Scheduling)
- RMS为静态优先级调度算法：为每个进程分配一个与事件发生频率成正比的优先级。例如周期为20ms的进程优先级为50，周期为100ms的进程优先级为10
- 运行时调度程序总是调度优先级最高的就绪进程，必要时将抢先当前进程

最早截止时间优先

- 最早截止时间优先(EDF, Earliest Deadline First)
- 一种动态优先级调度算法。当一个事件发生时, 对应的进程被加到就绪队列, 该队列按照截止时间排序。对于周期性事件, 截止时间即为事件下一次发生的时间
- 调度程序总是选择截止时间最早的进程

最小裕度优先

- 裕度(laxity)指进程的富裕时间，也称为松弛度(slack)
- $\text{裕度} = (\text{截止时刻} - \text{当前时刻}) - \text{剩余运行时间}$
- 例如一个进程需要运行100ms，它必须在200ms内完成，则在周期开始时刻执行其裕度为100ms，在周期开始后50ms执行其裕度为50ms
- 最小裕度优先(LLF, least laxity first)是一种动态优先级调度算法，它总是选择裕度最小的进程

最小裕度优先 vs 最早截止时间优先

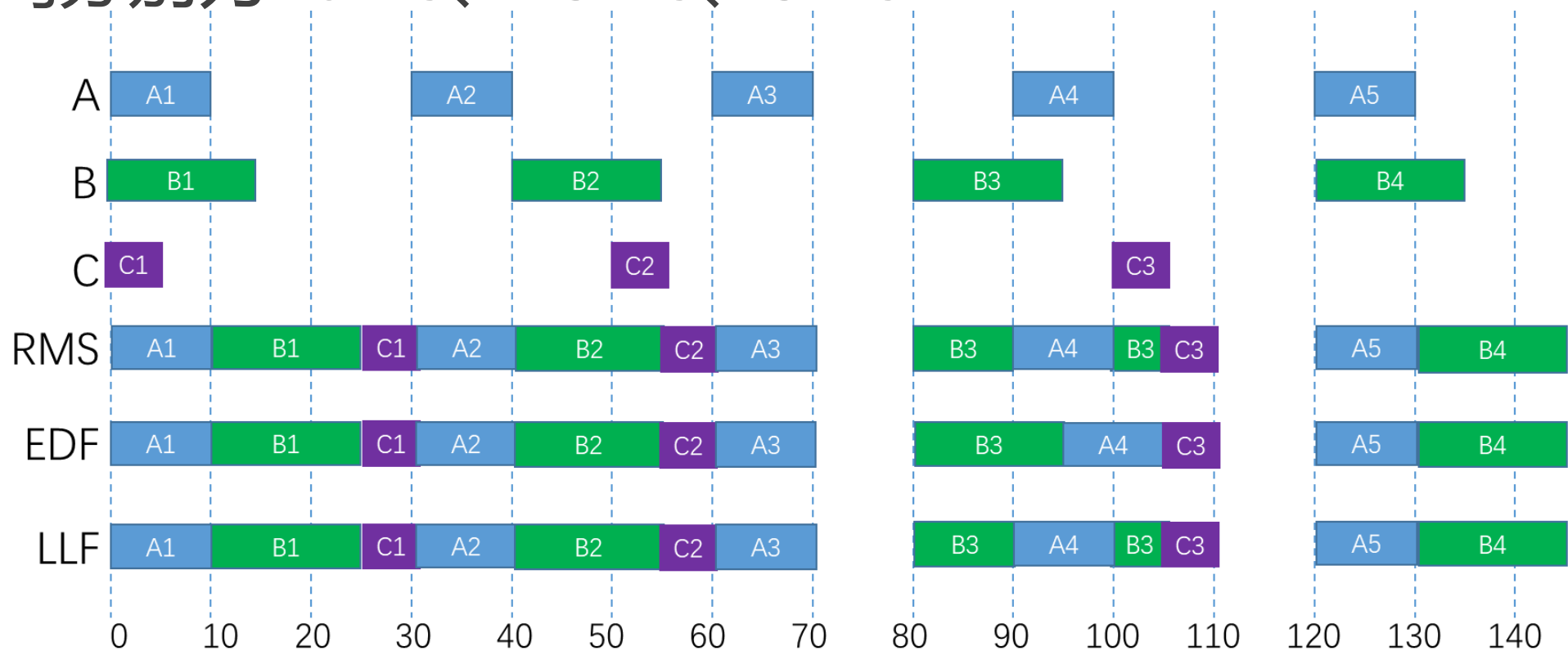
- EDF的紧急程度取决于截止时间早晚，谁的deadline最近，谁优先；LLF的紧急程度取决于还能拖多久，谁最禁不起拖，谁优先
- 例：当前时刻 $t=0$

Task	Deadline	Remaining Computation Time
A	4	1
B	6	4

- EDF调度A，LLF调度B

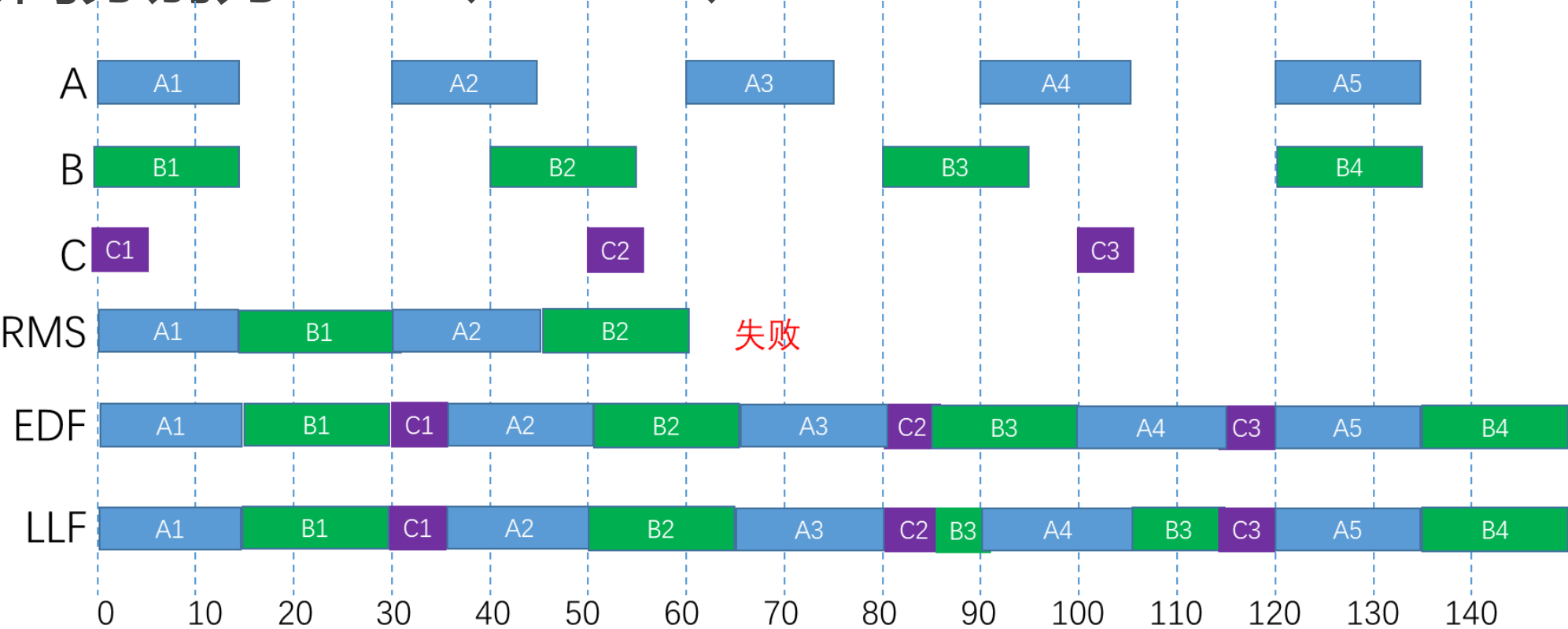
实时系统中的调度

- 进程A、B、C周期分别为30ms、40ms、50ms，每次运行所需时间分别为10ms、15ms、5ms



实时系统中的调度

- 进程A、B、C周期分别为30ms、40ms、50ms，每次运行所需时间分别为15ms、15ms、5ms



既考虑作业等待时间，又考虑作业执行时间的调度算法是

- ☐ A 先来先服务
- ☐ B 最短作业优先
- ☒ C 最高响应比优先
- ☐ D 最短剩余时间优先

提交

下述调度算法中，不会导致饥饿的是

- ☒ A 时间片轮转
- ☐ B 静态优先级调度
- ☐ C 非抢先式短作业优先
- ☐ D 抢先式短作业优先

提交

一个实时系统需要处理两个语音呼叫，每个呼叫每隔5ms发生一次，并且每次出现消耗1ms CPU时间。另外有一个25帧/秒的视频，每一帧需要20ms CPU时间。这个系统是可调度的吗？

☒ A 是

☐ B 否

大纲

01

处理机调度

02

Linux/Windows的处理机调度

02

死锁问题

Linux的处理机调度

- Linux内核主流版本（6.6+）是一个模块化的层级调度系统，是一个基于红黑树实现的最早合格虚拟截止日期优先 (Earliest Eligible Virtual Deadline First, EEVDF) 调度算法，辅以对称多处理 (SMP) 的负载均衡机制
- **Eligible（合格性）**：每个进程根据其权重应该分得一定的CPU时间。如果一个进程实际运行的时间少于它应得的时间，它就是合格的。只有合格的进程才有资格被选中运行，这保证了基本的公平性
- **Virtual Deadline（虚拟截止时间）**：调度器会为每个任务计算一个虚拟截止时间，这个时间代表了任务理应完成其当前时间片份额的时间点，它不仅仅考虑谁运行得少，还考虑谁的时间需求更紧迫

Linux的处理机调度

- EEVDF调度算法借鉴了经典实时调度算法EDF，在所有合格的任务中，调度器总是挑选那个虚拟截止时间最早的任务来运行
- EEVDF通过虚拟截止时间引入了对时间的敏感度，从而在保持整体公平的前提下，大幅降低交互式任务的响应延迟
- EEVDF的核心数据结构是红黑树
 - 左侧节点：vruntime(虚拟运行时间)较小的进程，代表它们运行得最少，最饥饿
 - 右侧节点：vruntime较大的进程，代表它们已经占用了较多资源
 - 调度动作：调度器每次都从树的最左侧取出任务执行。执行后，该任务的vruntime 增加，重新插入树中（通常会向右移动）

Linux的处理机调度

- 调度分级：Linux并不是单一调度算法，而是典型的模块化层级调度，优先级从高到低：
 1. Stop-Task： 极高优先级，用于CPU热插拔或迁移
 2. Deadline (SCHED_DEADLINE)： 基于 EDF，用于硬实时任务
 3. Real-Time (RT)： 包含 FIFO 和 Round Robin (轮转)，基于 0-99 的硬性优先级
 4. Fair (CFS/EEVDF)： 普通进程，即 SCHED_OTHER
 5. Idle： 仅在 CPU 完全空闲时运行

Linux的处理机调度

- 针对多核系统，Linux实现了负载均衡算法
- 运行队列 (Runqueue): 每个 CPU 都有自己的私有运行队列 (Runqueue)，减少锁竞争
- 调度域 (Scheduling Domains): 算法感知硬件拓扑 (如 SMT 超线程、多核、多插槽 NUMA)
- 推拉机制 (Push-Pull): 当一个 CPU 过载而另一个空闲时，内核会进行任务迁移，尽量保持各核心负载均衡

Windows的处理机调度

- Windows 11的调度算法是多级反馈队列MLFQ的变体，融合了对称多处理（SMP）下的亲和性（Affinity）策略以及针对混合架构（Hybrid Architecture）的硬件辅助调度
- 核心调度模型为基于优先级的抢占式多级反馈队列MLFQ，系统维护32个优先级队列（0-31）：
 - 16-31（实时优先级）：采用固定优先级，通常用于系统核心进程
 - 1-15级（动态优先级）：普通线程的优先级，调度器会根据线程行为动态调整其优先级

Windows的处理机调度

■ 反馈机制：

- I/O 提升： 当线程从 I/O 等待中返回时，调度器会暂时提升其优先级，使其更快获得 CPU 响应
- 前台提升： 用户当前交互的窗口会获得更大的时间片和优先级提升，以保证 UI 响应
- 饥饿补偿： 为了防止低优先级线程被饿死，调度器会扫描就绪队列，定期提升长时间未运行线程的优先级

Windows的处理机调度

- 在多核环境下，Windows 11 遵循经典 SMP 调度原则：
- 软亲和性（Soft Affinity）：调度器倾向于将线程分配回上次运行的核心，以减少 L1/L2 缓存缺失
- 理想处理器（Ideal Processor）：每个线程都有一个偏好的处理器，调度器在负载均衡时会优先考虑该处理器

Windows的处理机调度

- 针对Intel Thread Director (ITD)引入的性能核 (P-Core) 和能效核 (E-Core) 混合架构, 支持软硬件协同的非对称调度。性能核 (P-core) 擅长处理复杂的、对单线程性能要求高的任务, 能效核 (E-core): 擅长处理高并发的、对功耗敏感的背景任务
- 硬件监控: 处理器内部实时监控每个线程的指令组合 (是简单的后台任务还是复杂的矢量计算)
- 反馈调度: ITD 将这些实时数据反馈给 Windows 11 调度器
- 动态放置: 调度器根据反馈, 将高性能需求 (如游戏、生产力工具) 分配给 P-Core, 将后台任务 (如更新、杀毒软件) 引导至 E-Core, 并能根据负载变化在纳秒级时间内完成线程迁移

大纲

01

处理机调度

02

Linux/Windows的处理机调度

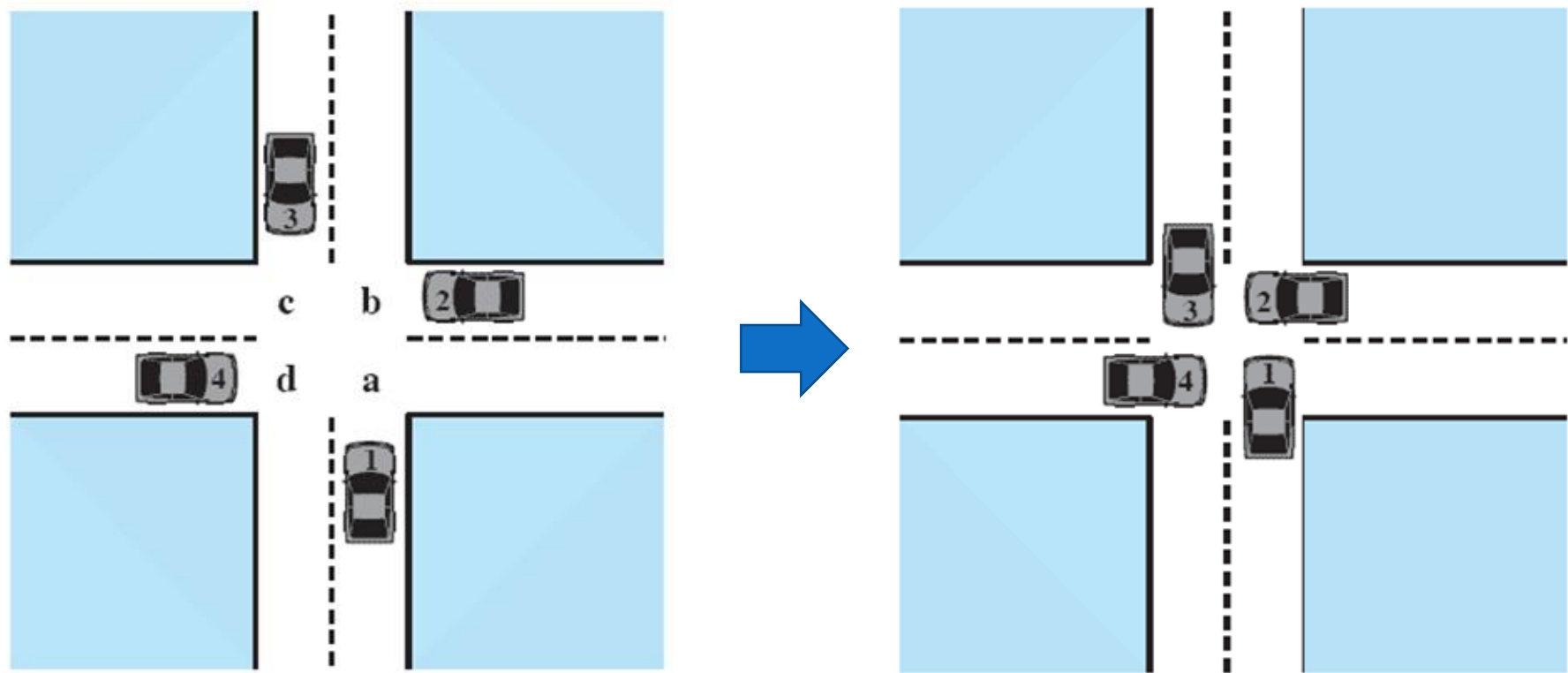
02

死锁问题

死锁问题

- 在多道程序系统中，多个进程并发运行，共享资源，从而提高了资源的利用率。但是若对资源的管理和使用不当，在一定条件下会导致系统发生一种随机性故障——死锁。发生死锁时，若无外力协助，每个进程都不能继续执行下去
- 死锁(Deadlock)是指系统中多个进程无限制地等待永远不会发生的条件
- 在一些系统（例如实时控制系统）中，一旦发生死锁将导致灾难性的后果

生活中的死锁问题



产生死锁的原因

- **竞争资源**：当系统中供多个进程所共享的资源，不足以同时满足它们的需要时，引起它们对资源的竞争而产生死锁
- **进程推进的顺序不当**：进程在运行过程中，请求和释放资源的顺序不当，导致进程的死锁

资源的分类

1. 根据资源性质：可抢占和不可抢占资源

- **可抢占资源**——指资源占有进程虽然需要使用该资源，但另一个进程强行把资源从占有者进程处抢来，而不会引起失败性后果
- CPU、主存、硬盘，可为几个进程共同使用——可抢占
- **不可抢占资源**——指只有占用者进程不再需要使用该资源而主动释放资源外，其它进程不得在占有者进程使用资源过程中强行抢占，否则会引起失败性后果
- 打印机、CD刻录机，只可为某个进程独享——不可抢占

资源的分类

2. 根据使用方式：共享资源和独占资源

3. 根据使用期限：永久资源（可重用性资源）和临时性资源（可消耗性资源）

- 永久资源：可重复使用的资源，例如所有的硬件资源和可再入的纯代码过程

- 临时性资源：由一个进程产生，被另外一个进程使用短暂时间之后便无用的资源，如在进程同步和通信中出现的消息、信号和数据

资源的分类

2. 根据使用方式：共享资源和独占资源

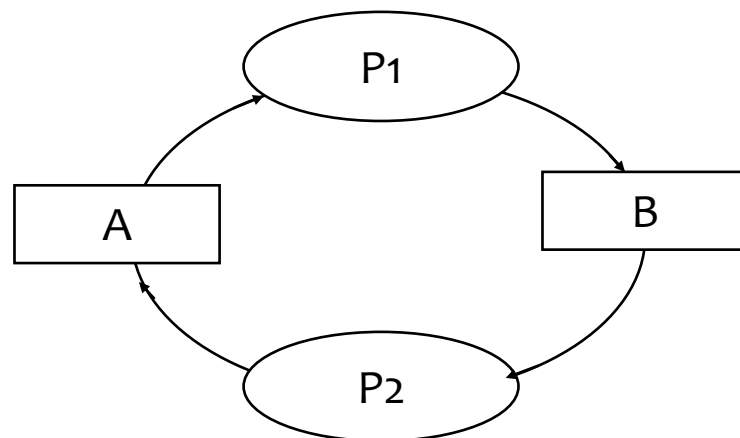
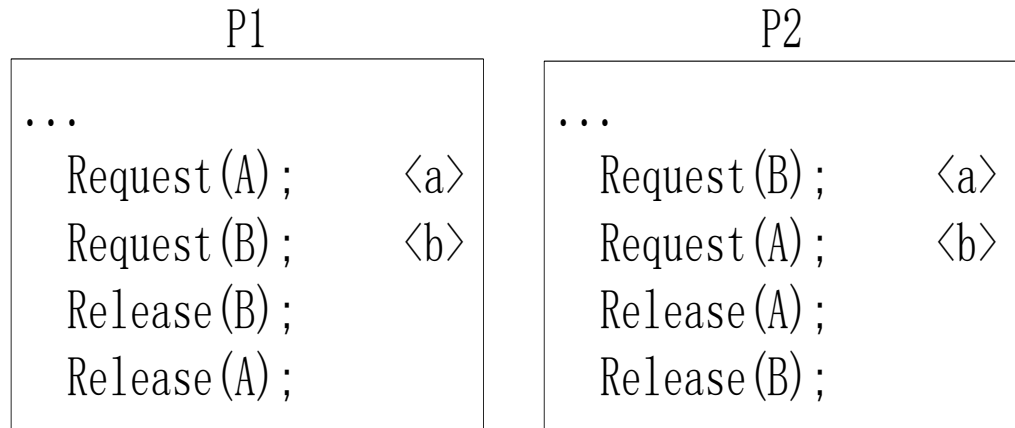
3. 根据使用期限：永久资源（可重用性资源）和临时性资源（可消耗性资源）

- 永久资源：可重复使用的资源，例如所有的硬件资源和可再入的纯代码过程

- 临时性资源：由一个进程产生，被另外一个进程使用短暂时间之后便无用的资源，如在进程同步和通信中出现的消息、信号和数据

竞争资源导致的死锁

- **竞争不可抢占资源**：双方都拥有部分资源，同时在请求对方已占有的资源。如次序：P1<a> P2<a> P1 P2



竞争资源导致的死锁

- **竞争临时性资源**：进程通信，双方都等待对方去生成资源，如
次序：P1<a> P2<a>

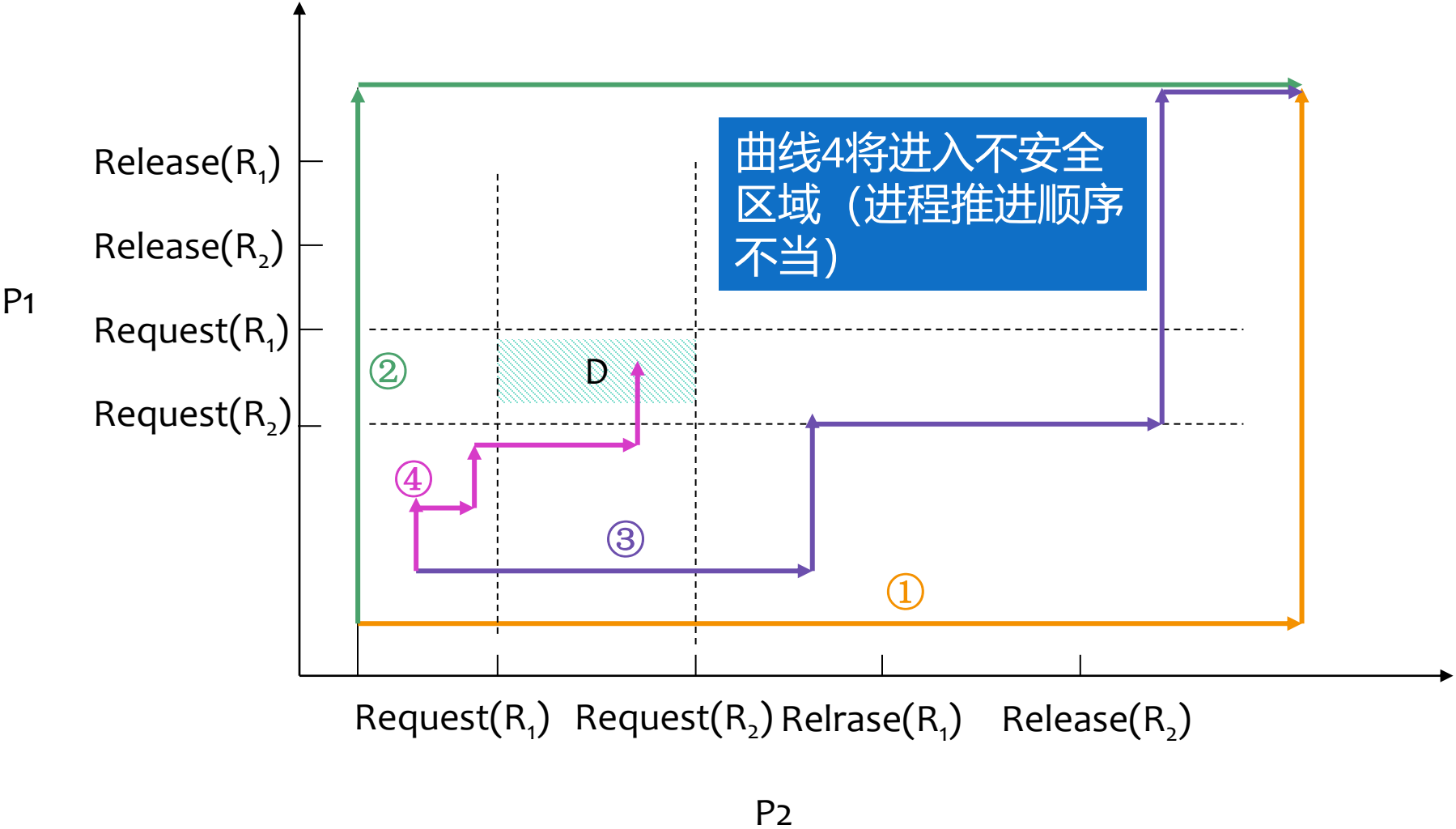
P1

```
...  
Receive (P2, M) ;    <a>  
Send (P2, N) ;      <b>
```

P2

```
...  
Receive (P1, Q) ;    <a>  
Send (P1, R) ;      <b>
```


进程推进的顺序不当导致的死锁



死锁发生的必要条件

- 互斥：任一时刻只允许一个进程使用资源
- 非抢占：进程已经占用的资源，不会被强制剥夺
- 请求和保持：进程在请求其余资源时，不主动释放已经占用的资源
- 环路等待：存在一种进程的循环链，链中的每一个进程已获得的资源同时被链中的下一个进程所请求

处理死锁问题的方法

- 鸵鸟算法
- 死锁预防
- 死锁避免
- 死锁检测

鸵鸟算法

- 传说中，鸵鸟看到危险就把头埋在沙子里，假装危险根本就不存在
- 当人们对某一件事情没有一个很好的解决方法时，那就忽略它，就像鸵鸟面对危险时会把它深埋在沙子中，装作看不到，这样的算法称为**鸵鸟算法(Ostrich algorithm)**
- 鸵鸟算法并不能算是一个算法，但却是目前实际系统采用最多的一种策略



鸵鸟算法

- 核心思想：忽略死锁问题，视而不见
- 原因：
 - 死锁问题的发生的概率很小
 - 处理死锁问题的代价非常大
- UNIX、Linux和Windows采用这方法

死锁预防

- 如果能够保证死锁的四个必要条件中至少有一个不满足，那么从结构上死锁将不会产生
- 死锁预防(**deadlock prevention**)就是采用某种策略，限制并发进程对资源的请求，使系统在任何时刻都不满足死锁的必要条件

破坏互斥条件

- 某些设备(例如打印机)可以假脱机操作
 - 只有打印机守护程序使用打印机资源
 - 这样就可以消除打印机死锁
- 不是所有的设备都可以进行假脱机操作

破坏非抢占条件

- 消除非抢占条件通常不是有效的
- 例如，如果一个进程已分配到一台打印机，且正在进行打印，如果由于它需要的绘图仪无法获得而强制性地把它占有的打印机剥夺掉，会引起一片混乱

破坏请求和保持条件

- **预先静态分配法**：进程开始运行前一次分配所需全部资源，若系统不能满足，则进程阻塞，直到系统满足其要求——保证进程运行过程中不会再提出新的资源请求
- **存在的问题**
 - 降低了对资源的利用率，降低进程的并发程度
 - 有可能无法预先知道所需资源
- **改进措施**：进程必须释放已分配的所有资源之后才能在需要时申请其他所需资源

破坏环路等待条件

- **有序资源使用法**：把资源分类按顺序排列，保证对资源的请求不形成环路
- **存在的问题**：
 - 限制进程对资源的请求顺序
 - 资源的排序占用系统开销

死锁避免

- 死锁避免(deadlock avoidance)并不事先采取限制去破坏产生死锁的条件，而是在资源的动态分配过程中，用某种方法去防止系统进入不安全状态，从而避免死锁的发生
- 优点：事先只需要较弱的限制条件，可获得较高的资源利用率和系统吞吐量
- 缺点：实现较难

银行家问题

- Dijkstra, 1965

- 一个银行家如何将一定数目的资金安全地借给若干个客户，使这些客户既能借到钱完成要干的事，同时银行家又能收回全部资金而不至于破产，这就是银行家问题
- 银行家问题同操作系统中资源分配问题十分相似：银行家就像一个操作系统，客户就像运行的进程，银行家的资金就是系统的资源

银行家问题

- 例如，一个银行家拥有1000万资金，有10家工厂筹建，且每家工厂需要200万方可建成。如何合理分配资金？
- 如果将1000万均分给10家，则每个工厂都无法建成，也不能还贷，也即这10家工厂将“死锁”
- 若给其中的三个工厂200万，另4家工厂100万，这样，虽然有3家工厂未开工，但有3家可以建成投产，这3家工厂获得的利润可以给银行还贷，银行家再利用还贷的资金继续给其它工厂投入，直至最终10家工厂都建成投产

银行家算法的思路

■ 银行家可以把一定数量的资金供多个用户使用，为保证资金的安全银行家规定四个条件：

1. 当一个用户对资金的最大需求量不超过银行家现有资金时就可接纳该用户
($\text{Request} \leq \text{Available}$)
2. 用户可以分期贷款，但贷款的总数不能超过最大需求量 ($\text{Request} \leq \text{Need}$)
3. 当银行家现有的资金不能满足用户的尚需贷款数时，对用户的贷款可推迟支付，但总能在用户有限的时间里得到贷款 (不符合“安全性条件”暂时不给贷款)
4. 当用户得到所需的全部资金后，一定能在有限的时间里归还所有的资金

死锁避免

- 在系统运行过程中，对进程提出的每一个（系统能够满足的）资源申请进行动态检查（安全性检查），并根据检查结果决定是否分配资源，若分配后系统可能发生死锁，则不予分配，否则予以分配
- **安全状态**：存在某种资源调度顺序，保证所有进程正常运行完成，则称该状态为安全状态
- **不安全状态**：不存在可满足所有进程正常运行的资源调度顺序，则称该状态为不安全状态
- 虽然并非所有不安全状态都是死锁状态，但当系统进入不安全状态后，便有可能进入死锁状态；反之只要系统处于安全状态，系统便可避免进入死锁状态。因此，避免死锁的实质是如何使系统不进入不安全状态

安全状态举例

- 假定系统有三个进程P1、P2、P3，共有12台磁带机。进程P1总共要求10台磁带机，P₂和P₃分别要求4台和9台。设在T₀时刻，进程P1、P2和P3已经获得5台、2台和2台，还有3台空闲没有分配

进程	最大需求	已分配	还需	可用
P1	10	5	5	3
P2	4	2	2	
P3	9	2	7	

- 在T₀时刻，系统是安全的，因为存在一个安全序列P2, P1, P3

由安全状态向不安全状态的转换

- 如果不按安全序列分配资源，则系统可能会由安全状态进入不安全状态
- 如在 T_0 以后，P3要求1台磁带机，若系统分给它1台，则系统进入不安全状态。因为其余2台分给P2，P2完成后，只能释放4台，这既不能满足P1（5台），也不能满足P3（6台），将导致死锁。可见当P3申请资源时，尽管系统中有资源也不能分给它

进程	最大需求	已分配	还需	可用
P1	10	5	5	3
P2	4	2	2	
P3	9	2	7	

进程	最大需求	已分配	还需	可用
P1	10	5	5	2
P2	4	2	2	
P3	9	3	6	

银行家算法中的数据结构

- 可利用资源向量`Available`：含有m个元素的数组，其中每个元素代表一类可利用资源的数目
- 初值为系统中所配置该类资源的总数
- 例如

R1	R2	R3
5	6	5

银行家算法中的数据结构

■ **最大需求矩阵Max**: $n*m$ 矩阵, 表示 n 个进程的每一个对 m 类资源的最大需求

■ 例如

	R1	R2	R3
P1	5	6	2
P2	3	3	1
P3	4	2	5
P4	3	3	2

银行家算法中的数据结构

■ **分配矩阵Allocation**: $n*m$ 矩阵, 表示每个进程已分配的每类资源的数目

■ 例如

	R1	R2	R3
P1	2	1	2
P2	1	2	1
P3	2	2	2
P4	1	3	2

银行家算法中的数据结构

- 需求矩阵Need: $n*m$ 矩阵, 表示每个进程还需要各类资源数
- 例如

	R1	R2	R3
P1	3	5	0
P2	2	1	0
P3	2	0	3
P4	2	0	0

银行家算法中的数据结构

■关系

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

银行家算法描述

■ 设 $Request_i$ 是进程 P_i 的请求向量，当进程 P_i 提出资源申请时，系统执行下列步骤：

1. 若 $Request_i[j] \leq Need[i,j]$ （请求小于需求），转2；否则错误返回
2. $Request_i[j] \leq Available[j]$ （请求小于库存），转3；否则进程等待
3. **试分配**：假设系统分配了资源，则有：
 $Available[j] := Available[j] - Request_i[j];$
 $Allocation[i,j] := Allocation[i,j] + Request_i[j];$
 $Need[i,j] := Need[i,j] - Request_i[j]$
4. 执行**安全性算法**，若系统新状态是安全的，则分配完成，若系统新状态是不安全的，则恢复原状态，进程等待

安全性算法

- 为进行安全性检查，定义数据结构：

int Work[m]; //动态可分配资源，即系统可提供给进程继续运行所需的各类资源，初值为Available

Boolean Finish[n]; //是否有足够的资源分配给进程，初值false

- m为资源种类数，n为进程的数量

安全性算法步骤

1. $Work[j] := Available[j];$
 $Finish[i] := false;$
2. 寻找满足下列条件的 i , 如果不存在, 则转4
 - a. $Finish[i] = false;$
 - b. $Need[i,j] \leq Work[j];$ (需求小于动态可分配资源)
3. 进程 i 获取资源, 然后执行完毕, 并释放资源:
 $Work[j] := Work[j] + Allocation[i,j];$
 $Finish[i] := true;$
 转(2)
4. 若对所有 i , $Finish[i] = true$, 则系统处于安全状态, 否则处于不安全状态

银行家算法运行实例

- 设系统有五个进程和三类资源，每类资源分别有10、5、7
- 在T0时刻资源分配情况

资源 \ 进程	Max			Allocation			Need		
	A	B	C	A	B	C	A	B	C
P0	7	5	3	0	1	0	7	4	3
P1	3	2	2	2	0	0	1	2	2
P2	9	0	2	3	0	2	6	0	0
P3	2	2	2	2	1	1	0	1	1
P4	4	3	3	0	0	2	4	3	1

Available		
A	B	C
3	3	2

银行家算法运行实例

■T0时刻可以找到一个安全序列P1, P3, P4, P2, P0，系统是安全的

资源 进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P1	3	3	2	1	2	2	2	0	0	5	3	2	true
P3	5	3	2	0	1	1	2	1	1	7	4	3	true
P4	7	4	3	4	3	1	0	0	2	7	4	5	true
P2	7	4	5	6	0	0	3	0	2	10	4	7	true
P0	10	4	7	7	4	3	0	1	0	10	5	7	true

银行家算法运行实例

■T0时刻P₁发出请求Request(1, 0, 2)，执行银行家算法

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	7	5	3	0	1	0	7	4	3	3	3	2
P ₁	3	2	2	2	0	0	1	2	2	2	3	0
P ₂	9	0	2	3	0	2	6	0	0			
P ₃	2	2	2	2	1	1	0	1	1			
P ₄	4	3	3	0	0	2	4	3	1			

银行家算法运行实例

■ 执行安全性算法，可以找到一个安全序列P1, P3, P4, P0, P2，系统是安全的，可以将P1请求资源分配给它

资源 进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₁	2	3	0	0	2	0	3	0	2	5	3	2	true
P ₃	5	3	2	0	1	1	2	1	1	7	4	3	true
P ₄	7	4	3	4	3	1	0	0	2	7	4	5	true
P ₀	7	4	5	7	4	3	0	1	0	7	5	5	true
P ₂	10	4	7	6	0	0	3	0	2	10	5	7	true

银行家算法运行实例

- P4发出请求Request(3, 3, 0)，执行银行家算法
- Available=2 3 0
- 不能通过算法第2步 ($Request_i[j] \leq Available[j]$) ，所以P4等待

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	7	5	3	0	1	0	7	4	3	2	3	0
P ₁	3	2	2	3	0	2	0	2	0			
P ₂	9	0	2	3	0	2	6	0	0			
P ₃	2	2	2	2	1	1	0	1	1			
P ₄	4	3	3	0	0	2	4	3	1			

银行家算法运行实例

- P0发出请求Request(0, 2, 0)，执行银行家算法
- Available{2,1,0}已不能满足任何进程需要，所以系统进入不安全状态，P0的请求不能分配

资源 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	7	5	3	0	1	0	7	4	3	2	3	0
				0	3	0	7	2	0	2	1	0
P ₁	3	2	2	3	0	2	0	2	0			
P ₂	9	0	2	3	0	2	6	0	0			
P ₃	2	2	2	2	1	1	0	1	1			
P ₄	4	3	3	0	0	2	4	3	1			

银行家算法的优点

- 银行家算法允许死锁必要条件中的互斥条件、请求和保持和非剥夺条件存在，比起预防死锁的几种方法来说，限制条件放松，资源利用程度提高
- 由于系统可以满足、也可以拒绝进程提出的资源请求，当一个进程的资源要求将导致不安全状态时，系统就拒绝其要求，直到该资源要求不会导致不安全状态时，才满足此进程的资源要求(这主要由于其它进程释放出了资源)，这样系统总是处于安全状态

银行家算法的缺点

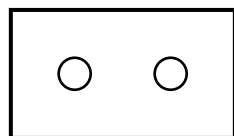
- 要求被分配的每类资源的数量是固定不变的，难以做到，因为资源本身需要维护，可能损坏
- 要求用户数也保持固定不变，在多道程序系统中难以做到的，用户数是经常变化的(尤其在分时环境中)

死锁的检测和恢复

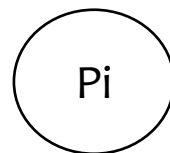
- 一般的系统着眼于提高资源利用率，会允许死锁发生的必要条件发生，也不采取避免死锁算法。这些系统可能发生死锁
- 死锁检测算法常被使用在允许前三个死锁必要条件存在的系统。死锁检测算法主要检查系统中是否存在循环等待条件
- 一旦发现这种环路便认定死锁存在，并识别出该环路所涉及的有关进程，以供系统采取适当的措施来解除死锁
- 最常用的是一种基于资源分配图和死锁定理的检测死锁算法

资源分配图 (RAG)

- 资源分配图由一组结点和一组边所组成
- 结点分为两类：
 - 资源用方框表示，资源个数（存在于每个资源类中）用方框中的圆圈表示
 - 进程用圆圈中加进程名表示



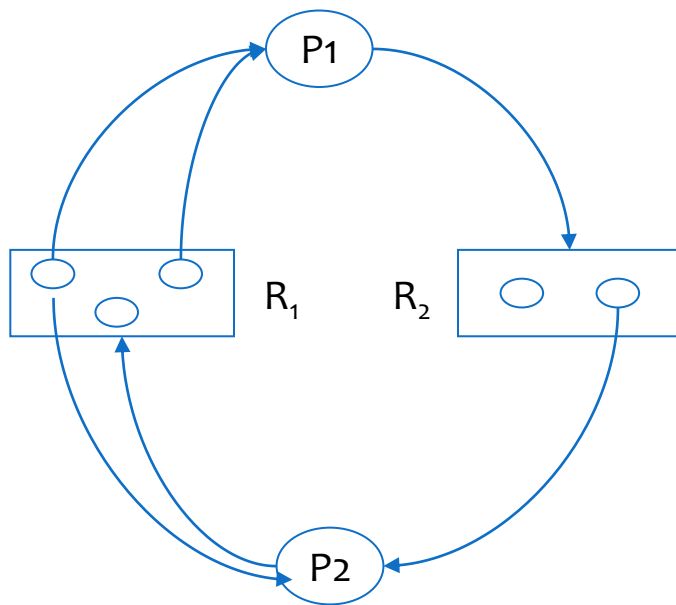
资源类



进程

资源分配图 (RAG)

- 分配边：资源实例→进程的一条有向边
- 申请边：进程→资源类的一条有向边



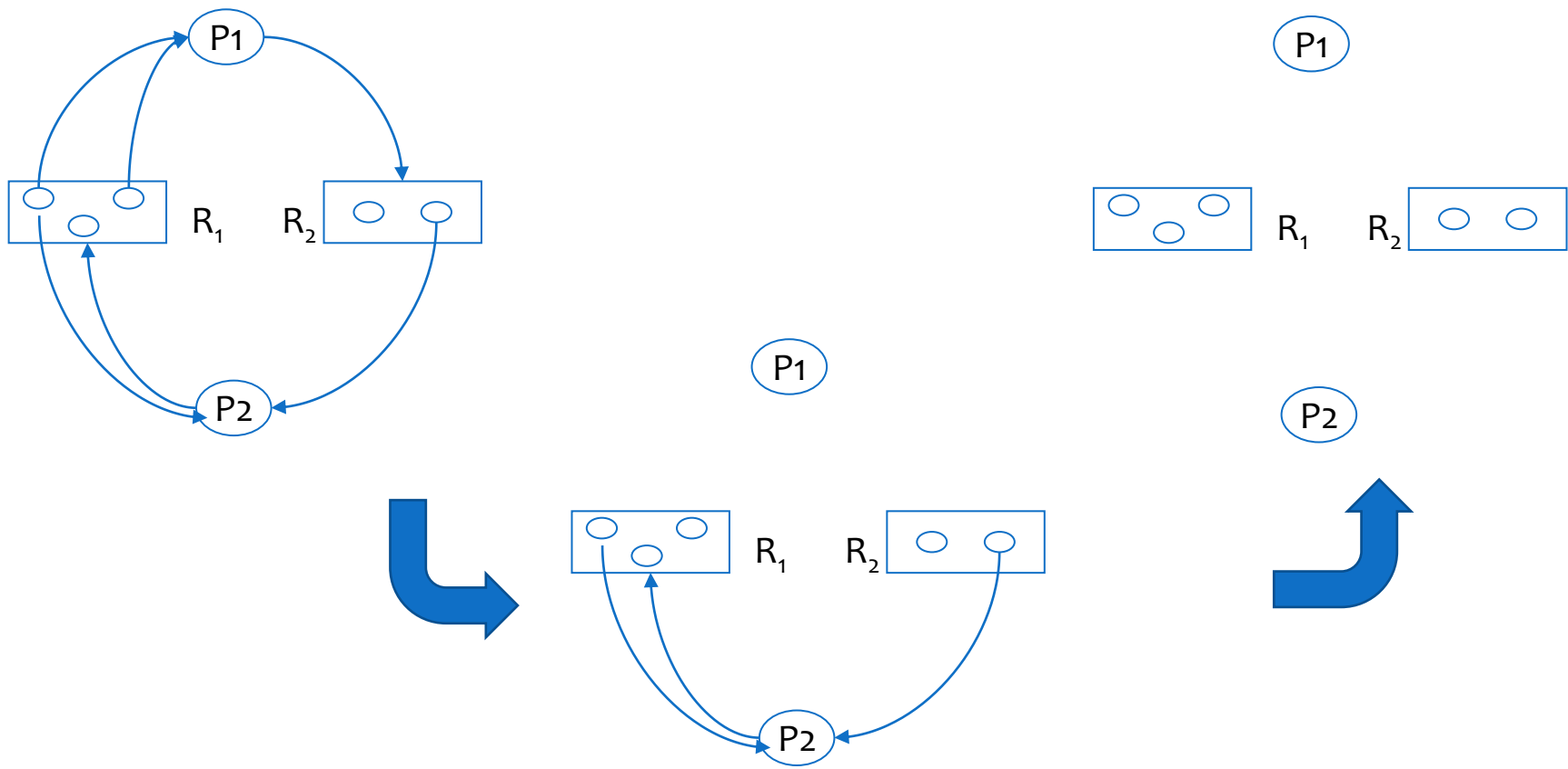
死锁检测

- 由于系统状态与资源分配图一一对应，所以对死锁的检测可以通过化简资源分配图的方式来实现
- 死锁的产生必然是由于资源分配图中出现了环，通过化简，就是要检查资源分配图中是否存在环，如果图中存在环路则系统中可能存在死锁

资源图的化简

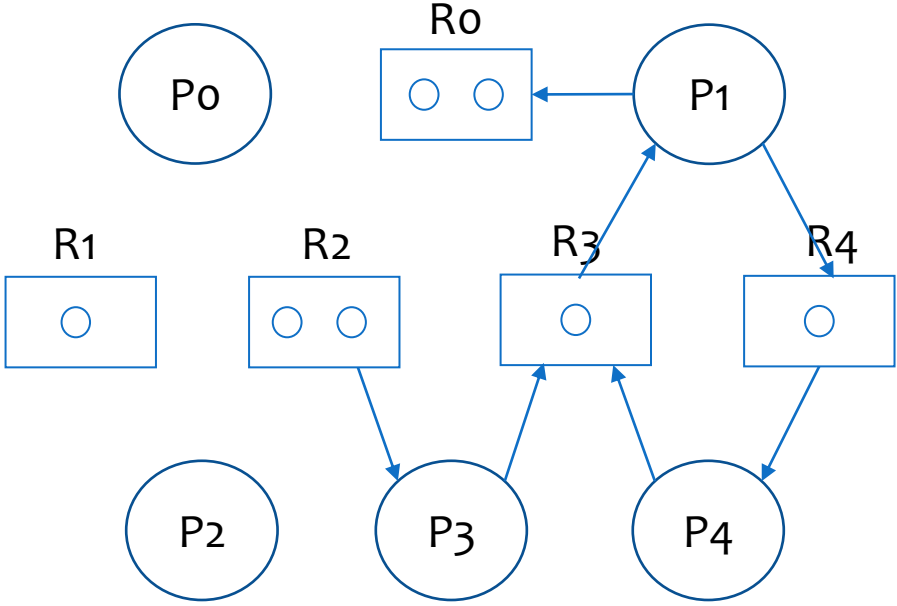
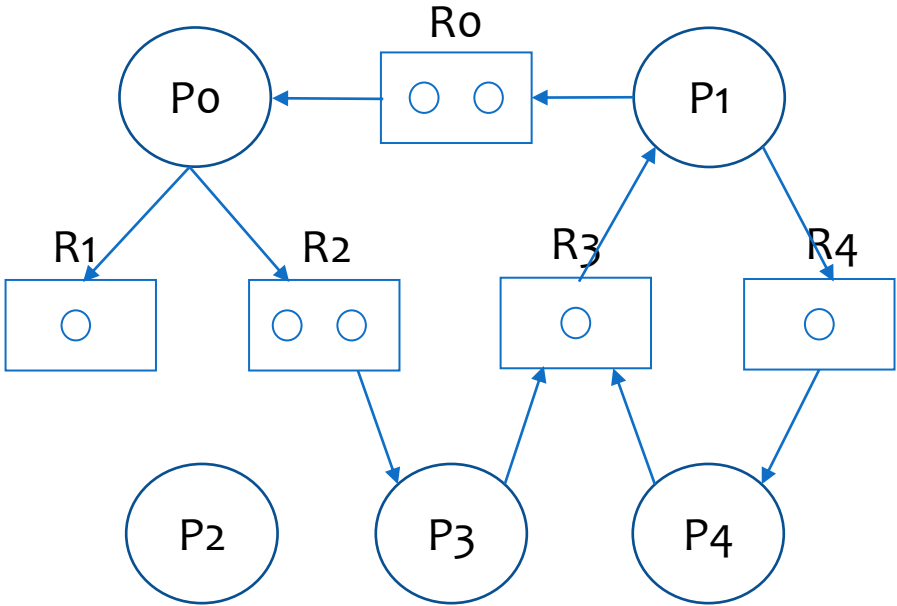
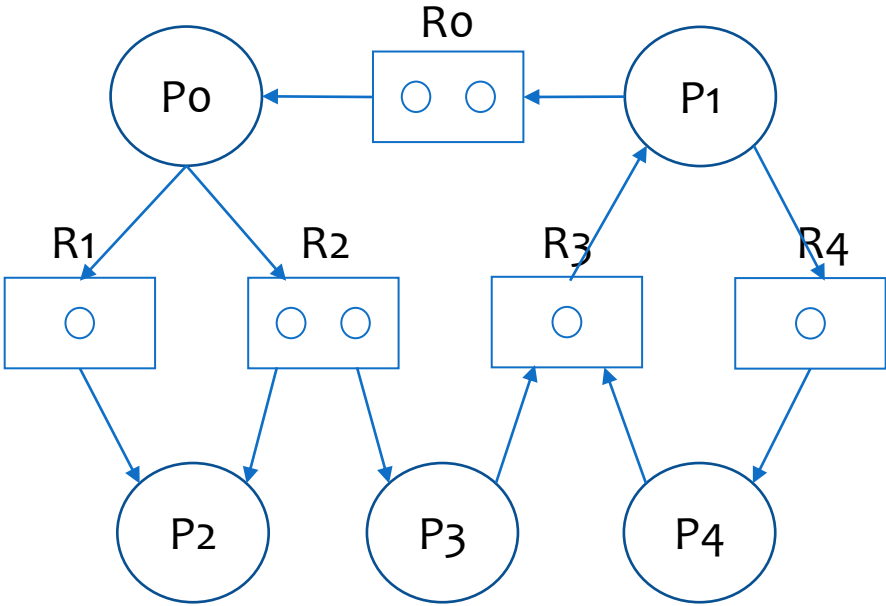
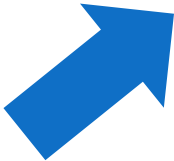
1. 找出一个既不阻塞又非孤立的进程结点 P_i ，消去 P_i 所有的请求边和分配边，使之成为孤立结点（意义：进程 P_i 可获得所需资源并运行完毕，然后释放占有的所有资源）
2. 循环执行1（意义：下一个进程在前面的进程释放资源后，可获得资源并执行完毕，然后又释放占有的全部资源）
3. 重复执行，直到所有进程节点都成为孤立结点，则该图是可化简的；否则，存在环，该图是不可化简的

资源图的化简



资源图的化简

不可化简图



死锁定理

- **引理**：对于给定的资源分配图，所有的化简顺序均可得到相同的化简结果图，即化简结果图与化简顺序无关
- **死锁定理**：死锁状态的**充分必要条件**是系统的资源分配图是不可完全化简的

死锁的解除

- 发现进程死锁时，从死锁中解脱出来的最常用的方法有两种
- 剥夺资源法**：从其它进程剥夺足够数量的资源给死锁进程，以解除死锁状态
- 撤消进程法**：撤消全部死锁进程，或是按照某种顺序逐个地撤消进程，直至有足够的资源可用，死锁状态消除为止

某计算机系统有8台打印机，有K个进程，每个进程最多使用3台打印机，该系统可能发生死锁的K的最小值是___。

A 1

C 3

B 2

☒ D 4

设系统有五个进程P0~P4和三类资源A、B、C，每类资源数分别为17、5、20。T0时刻，系统状态如下，系统是否安全？

☒ A 是 ☐ B 否

资源 进程	Max			Allocation			Need		
	A	B	C	A	B	C	A	B	C
P0	5	5	9	2	1	2	3	4	7
P1	5	3	6	4	0	2	1	3	4
P2	4	0	11	4	0	5	0	0	6
P3	4	2	5	2	0	4	2	2	1
P4	4	2	4	3	1	4	1	1	0

Available		
A	B	C
2	3	3

操作系统，2026

谢谢

OPERATING SYSTEM